

3. Longest Substring Without Repeating Characters Medium

Acceptance: 38.3% | [LeetCode](#)

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

```
**Input:** s = "abcabcbb"
**Output:** 3
**Explanation:** The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
```

Example 2:

```
**Input:** s = "bbbbbb"
**Output:** 1
**Explanation:** The answer is "b", with the length of 1.
```

Example 3:

```
**Input:** s = "pwwkew"
**Output:** 3
**Explanation:** The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- `s` consists of English letters, digits, symbols and spaces.

Claude Opus Solution

Longest Substring Without Repeating Characters

Intuition

The key insight here is that this is a classic **sliding window** problem. We need to find a contiguous substring (window) where all characters are unique.

Key Pattern Recognition: - When we encounter a duplicate character, we don't need to restart from scratch - We can "slide" our window by moving the left boundary past the previous occurrence of the duplicate - Using a hash map to store character positions allows $O(1)$ lookups for duplicates

Think of it as maintaining a "valid window" that expands when possible and contracts when we hit a duplicate.

Approach

1. Initialize:
 - A hash map to store each character's most recent index
 - A left pointer (start of window) at 0
 - A variable to track maximum length found
2. Iterate through string with right pointer (end of window):
 - If current character exists in map AND its stored index $\geq$ left pointer:
 - This means it's within our current window (duplicate found!)
 - Move left pointer to one position after the duplicate
 - Otherwise:
 - Character is unique in current window
 - Update `max_length` if current window is larger
 - Always update the character's position in the map
3. Return `max_length`

Python Solution

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # Hash map: character -> its most recent index in the string
        char_index = {}

        # Left boundary of our sliding window
        left = 0

        # Track the maximum length found so far
        max_length = 0
```

```

# Right boundary expands as we iterate through the string
for right, char in enumerate(s):
    # Check if this character creates a duplicate within our window
    # Two conditions must BOTH be true for it to be a "real" duplicate:
    # 1. Character exists in our map (we've seen it before)
    # 2. Its previous position is >= left (it's within current window)
    if char in char_index and char_index[char] >= left:
        # Duplicate found! Shrink window from left
        # Move left pointer just past the previous occurrence
        left = char_index[char] + 1
    else:
        # No duplicate - this window is valid
        # Current window size = right - left + 1
        max_length = max(max_length, right - left + 1)

    # Always update/record current character's position
    # This overwrites any previous position (we only care about most recent)
    char_index[char] = right

return max_length

```

Walkthrough Example

For `s = "abcabcbbb"`:

```

right=0, char='a': No dup, window="a", max=1, map={'a':0}
right=1, char='b': No dup, window="ab", max=2, map={'a':0,'b':1}
right=2, char='c': No dup, window="abc", max=3, map={'a':0,'b':1,'c':2}
right=3, char='a': Dup! 'a' at index 0 >= left(0), move left to 1
                  window="bca", map={'a':3,'b':1,'c':2}
right=4, char='b': Dup! 'b' at index 1 >= left(1), move left to 2
                  window="cab", map={'a':3,'b':4,'c':2}
... and so on

```

Complexity Analysis

Time Complexity: $O(n)$ - We iterate through the string exactly once with the right pointer - Each character is processed once (no nested loops) - Hash map operations (insert/lookup) are $O(1)$ average

Space Complexity: $O(\min(m, n))$ - Where m is the size of the character set and n is string length - In worst case, we store all unique characters - For ASCII: $O(128)$, for extended ASCII: $O(256)$ - More generally: $O(\min(\text{alphabet_size}, n))$

Alternative Approaches

1. Basic Sliding Window with Set (Solution 2 in reference)

```

def lengthOfLongestSubstring(self, s: str) -> int:
    char_set = set()
    left = max_length = 0

    for right in range(len(s)):
        # Remove characters from left until no duplicate
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        max_length = max(max_length, right - left + 1)

    return max_length

```

- **Time:** $O(2n) = O(n)$ - each character visited at most twice
- **Space:** $O(\min(m, n))$
- Simpler but slightly less efficient due to incremental left movement

2. Brute Force (Not recommended)

- Check all substrings for uniqueness
- **Time:** $O(n^3)$ - $O(n^2)$ substrings $\times O(n)$ to check each

- **Space:** $O(n)$

The hash map approach (Solution 1) is optimal because it allows us to "jump" the left pointer directly to the correct position instead of incrementing one by one.

GPT-5 Solution

1) Intuition (Key Insight / Pattern)

This is a classic **sliding window** problem:

- We want the longest **contiguous** substring with **all unique** characters.
- If a window `[L..R]` has no duplicates, expanding `R` is always good (may increase answer).
- When adding `s[R]` creates a duplicate, the window becomes invalid. The only way to fix it while keeping contiguity is to **move L right** until the duplicate is removed.

Key trick to do this efficiently: - Track the **last index where each character appeared** (`last[ch]`). - When we see a repeated character `ch` at index `R`, we can jump `L` directly to `last[ch] + 1` (but never move `L` backwards).

This makes the solution linear time.

2) Approach (Step-by-step Pseudocode)

```
function longestUniqueSubstringLength(s):
    last = empty map // char -> last seen index
    L = 0
    ans = 0

    for R from 0 to len(s)-1:
        ch = s[R]

        if ch in last AND last[ch] >= L:
            // ch repeated inside current window
            L = last[ch] + 1

        last[ch] = R
        ans = max(ans, R - L + 1)

    return ans
```

3) Python Solution (Heavily Commented)

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # last[c] = the most recent index where character c was seen
        last = {}

        # L = left boundary of the current sliding window [L..R]
        L = 0

        # best answer found so far
        best = 0

        # Expand the window by moving R from left to right
        for R, ch in enumerate(s):
            # If we have seen ch before AND its last position is inside the current window,
            # then ch is a duplicate within [L..R].
            # We must move L to one position right of that last occurrence to restore uniqueness.
            #
            # Important: we only update L if last[ch] >= L
            # because if last[ch] < L, that occurrence is already outside the window.
            if ch in last and last[ch] >= L:
                L = last[ch] + 1

            # Update the last seen index of this character to the current position
            last[ch] = R
```

```
# Now window [L..R] is guaranteed to have all unique characters
best = max(best, R - L + 1)

return best
```

4) Complexity Analysis

- **Time Complexity: $O(n)$**
- Each character is processed once as `R` moves forward.
- `L` only moves forward (never backward), and we “jump” it using `last`, so total movement is also $O(n)$.
- Hash map operations are amortized $O(1)$.
- **Space Complexity: $O(k)$**
- `last` stores at most one entry per distinct character.
- In worst case, $k = \min(n, \text{charset_size})$. For general ASCII-like input, it’s bounded; for arbitrary chars, it can be up to `n`.

5) Alternative Approaches (Brief)

1. **Sliding Window with a Set (two pointers)**
2. Maintain a set of chars in the window.
3. If duplicate appears, move `L` one step at a time removing chars until valid.
4. Still $O(n)$ time, but may do more incremental steps than the “jump” method.
5. **Brute Force**
6. Check all substrings and test uniqueness.
7. Typically $O(n^3)$ (or $O(n^2)$ with optimization), too slow for `n = 5e4`.
8. **Fixed-size array instead of dict**
9. If character set is known small (e.g., ASCII 128/256), use an integer array for last positions for speed and predictable memory.